

Архиватор на базе алгоритма Хаффмана

Лиханов Даниил Викторович

Группа: 25.Б73-мм

Кафедра: кафедра системного программирования

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

1. Введение

Данный проект представляет собой учебную задачу по разработке консольного приложения-архиватора на основе алгоритма Хаффмана. Работа позволяет на практике освоить механизмы побитового ввода-вывода, методы построения префиксных кодов переменной длины и принципы управления динамическими структурами данных (бинарными деревьями) на языке C++.

2. Постановка задачи

Актуальность работы обусловлена необходимостью получения практических навыков низкоуровневой обработки информации. Умение работать с битовыми кодами и побитовым вводом-выводом, освоенное в ходе реализации алгоритма Хаффмана, имеет высокую практическую ценность и востребовано в различных актуальных областях программной инженерии, требующих эффективного управления памятью и ресурсами. При этом сам алгоритм Хаффмана является одним из наиболее известных методов кодирования и широко применяется в различных форматах хранения и передачи данных. Целью работы является разработка консольного приложения, использующего алгоритм Хаффмана для уменьшения объёма хранимой информации без потери данных.

Для достижения цели требуется решить следующие задачи:

1. Выполнить обзор существующих решений и изучить теоретические основы алгоритма Хаффмана.
2. Спроектировать архитектуру приложения и его основные модули.
3. Реализовать программный код для сжатия и восстановления исходных файлов.
4. Провести тестирование разработанного решения на файлах различного типа и размера.

3. Обзор

Сжатие без потерь (*lossless compression*) применяется там, где критически важна абсолютная точность восстановления исходных данных (тексты, программный код, исполняемые файлы). В рамках данного проекта рассматриваются теоретические основы и подходы к реализации алгоритма частотного кодирования Хаффмана.

3.1. Теоретические основы алгоритма Хаффмана

Метод основан на подсчёте частоты появления каждого символа в файле. Символам, которые встречаются чаще всего, присваиваются самые короткие битовые коды, а редким символам — более длинные. Связь между символами и кодами строится с помощью бинарного дерева. При этом построенные коды обладают важным свойством префиксности (ни один код символа не является началом (префиксом) кода другого символа). Например, если символ «А» закодирован как 01, то ни один другой символ не может иметь код, начинающийся с 01. Главная сложность и особенность классического алгоритма заключается в необходимости сохранять структуру дерева или таблицу частот внутри сжатого файла, чтобы принимающая программа (декодер) могла правильно восстановить данные.

3.2. Подходы к реализации

Существует множество открытых программных реализаций алгоритма Хаффмана, которые различаются выбранными структурами данных, такими как [1] для построения дерева кодирования используется очередь с приоритетами: на каждом шаге извлекаются два узла с наименьшим весом и объединяются в новый родительский узел. Для построения дерева кодирования классическим подходом является использование очереди с приоритетами, что позволяет эффективно извлекать два узла с минимальным весом и объединять их, создавая нового общего родителя.

4. Описание предлагаемого решения

Программа написана на языке C++ (стандарт C++17).

Логика программы разделена на четыре последовательных шага:

1. **Подсчёт частот:** Чтение файла и сбор статистики появления каждого байта.
2. **Построение дерева:** Создание бинарного дерева Хаффмана на основе полученных частот.
3. **Создание таблицы кодов:** Обход дерева от корня к листьям для генерации новых битовых кодов.
4. **Битовый ввод-вывод:** Упаковка данных в бинарный файл архива с помощью специального побитового буфера.

Наибольшую сложность в проекте вызвало управление памятью при построении дерева Хаффмана. Алгоритм требует постоянно извлекать из очереди два узла с минимальным весом, объединять их и создавать для них нового общего родителя.

5. Эксперименты

Для оценки производительности и эффективности **алгоритма Хаффмана** был написан специальный тестовый модуль на языке C++. В качестве исходных данных использовался набор из шести файлов, представляющих различные типы информации. Текстовые данные разного объёма: `small.txt` (содержит краткое содержание романа Л. Н. Толстого «Война и мир») и `large.txt` (содержит полный текст романа «Война и мир»). Несжатые бинарные данные: `binary.bin` (скомпилированный 64-битный исполняемый файл для ОС Linux в формате ELF). Предварительно сжатые мультимедийные форматы и архивы: `compressed.zip`, `image.jpg`, `music.mp3`. Все тестовые файлы доступны в открытом репозитории проекта [2].

Для получения статистически достоверных результатов замеры времени проводились программно. Все эксперименты проводились на следующей аппаратной и программной конфигурации:

- 1. Процессор: Intel Core i5-13420H (12 ядер, базовая частота ~2.1 ГГц).
- 2. Оперативная память: 16 ГБ DDR4.

Программная среда:

- 1. ОС: Windows 11 Pro 64-разрядная.
- 2. Среда выполнения: WSL2 (Windows Subsystem for Linux 2).
- 3. Ядро Linux: 6.6.87.2-microsoft-standard-WSL2.
- 4. Дистрибутив Linux: Ubuntu 24.04.1 LTS.

Инструменты сборки и компиляции:

- 1. Компилятор: g++ 13.3.0
- 2. Стандарт языка: C++17
- 3. Система сборки: CMake 3.20.
- 4. Конфигурация сборки: Release.
- 5. Флаги компиляции: `-std=c++17 -O2 -march=native`.

Каждая операция (**сжатие** — `encodeFile` и **распаковка** — `decodeFile`) выполнялась в цикле по 10 раз для каждого файла. Замер времени осуществлялся с помощью библиотеки `<chrono>` [3].

На основе массива полученных временных интервалов программа автоматически рассчитывала:

- 1. Среднее время выполнения операции (математическое ожидание).
- 2. Среднеквадратичное отклонение (СКО) для оценки стабильности работы алгоритма.

5.1. Результаты экспериментов

В **таблице 1** представлены сводные результаты работы программы (усреднённые данные за 10 прогонов). Коэффициент эффективности сжатия вычислялся как процентное отношение уменьшенного размера к оригинальному.

Таблица 1: Сводные результаты работы программы

Файл	Исходный размер (байт)	Сжатый размер (байт)	Эффективность сжатия	Время сжатия, среднее ± СКО (мс)	Время распаковки, среднее ± СКО (мс)
<code>small.txt</code>	10 343	7 290	+29.52%	2.47 ± 0.26	2.16 ± 0.15
<code>large.txt</code>	1 531 380	952 237	+37.82%	47.64 ± 5.94	103.74 ± 2.22
<code>binary.bin</code>	312 688	228 583	+26.90%	11.00 ± 0.35	24.40 ± 2.17
<code>compressed.zip</code>	4 588 359	4 560 423	-0.05%	138.18 ± 3.22	394.27 ± 4.34
<code>image.jpg</code>	2 054 988	2 056 590	-0.08%	62.40 ± 6.28	199.09 ± 4.13
<code>music.mp3</code>	10 553 390	10 508 615	+0.43%	335.68 ± 8.95	911.20 ± 440.13

Выводы. Алгоритм демонстрирует высокую эффективность при работе с текстовыми данными. Например, файл `large.txt` объёмом почти 1.5 МБ был сжат до 0.9 МБ. При обработке уже сжатых форматов (`.zip`, `.jpg` и `.mp3`) размер файла ожидаемо остался неизменным или незначительно увеличился. Результаты эксперимента, проведённого в автоматическом режиме на серии из 10 запусков, подтверждают, что **алгоритм Хаффмана** показывает высокую эффективность на избыточных данных (текстах) и демонстрирует предсказуемое, стабильное сжатие текстовых файлов.

6. Заключение

В ходе выполнения работы получены следующие результаты:

- 1. Спроектировано и реализовано программное решение на языке C++ для сжатия и точного восстановления файлов с использованием алгоритма префиксного кодирования Хаффмана (модули `encoder` и `decoder`).
- 2. Разработан автоматизированный модуль тестирования для оценки скорости и стабильности работы реализованных алгоритмов.
- 3. Проведена успешная экспериментальная проверка программы на наборе файлов различных форматов и объёмов.

Список литературы

- [1] Nayuki. Reference Huffman Coding [Электронный ресурс]. — URL: <https://www.nayuki.io/page/reference-huffman-coding> (дата обращения: 14.06.2026).
- [2] Репозиторий с исходным кодом проекта [Электронный ресурс]. — URL: <https://github.com/liptonn405/huffman-archive> (дата обращения: 14.06.2026).
- [3] C++ Reference: std::chrono library [Электронный ресурс]. — URL: <https://en.cppreference.com/w/cpp/chrono> (дата обращения: 14.06.2026).